1 Service overview

Definition: `plantDescriptionManagement`, version 1.0.0. Produced by `PlantDescriptionEngine`; HTTPS uses `generic_https` and `CERT_AUTH`. Consumption authorization and management restrictions are additionally enforced at the application layer. Base path: `/plantdescriptionengine/mgmt`.

2 Operations

create-plant-description `POST /plant-descriptions`.

get `GET /plant-descriptions/{id}`.

get-plant-description `GET /plant-descriptions/{id}`.

query `GET /plant-descriptions`.

query-plant-descriptions `GET /plant-descriptions`.

remove-plant-description `DELETE /plant-descriptions/{id}`.

replace-plant-description `PUT /plant-descriptions/{id}`.

sync-status `GET /sync-status`.

update-plant-description `PATCH /plant-descriptions/{id}`.

2.1 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Topology/include composition, named and metadata matching, periodic alarms, and atomic PDE-to-DSO snapshot synchronization are implemented. Missing producers retain managed empty scopes; nonnegative priorities rank eligible providers. Java SQL import, complete DTO normalization and production qualification remain outside this extension.

3 Interface contract

See the system HTTPS IDD and the Java source inventory for DTO fields and reference traceability. Successful local tests do not establish all Java behavioral or failure semantics.

4 PDE and DSO snapshot extension (23 October 2026)

The local Java, Go and native Rust implementations provide the same HTTPS snapshot contract. The registered service is `serviceOrchestrationPlantManagement`, version 1.0.0. The API is a local extension to the upstream framework.

Method	Resource and operation
POST	`/serviceorchestration/mgmt/plant-rules/query` (query)
PUT	`/serviceorchestration/mgmt/plant-rules/replace` (replace)

Both operations return the complete snapshot. The request for replacement has `rules`, an array of scopes. Each scope contains `consumer`, `serviceDefinition`, and `providers`; each provider contains `serviceInstanceId` and integer `priority`. Priority zero is valid; lower values rank first. Duplicate scopes and provider IDs, null arrays, and invalid priorities are rejected before commit. Empty rules clear management; empty providers retain a managed but unavailable scope.

PDE composes includes and resolves names/metadata against registered systems and instances. Named consumers remain managed when unregistered; missing or ambiguous producers retain empty scopes. Repeated provider IDs use the minimum priority. PDE queries before replacing and requires an equal acknowledgment. Failed reads do not clear rules. Registry query pages are bounded at 1000 entries. An incomplete response fails synchronization and preserves the last DSO snapshot. Changes reconcile normally within two seconds of a plant change and within thirty seconds of registry changes. `/plantdescriptionengine/mgmt/sync-status` exposes the loop state under management access.

DSO restricts candidates to the scope, retains existing authorization, operation/interface and exclusivity checks, then ranks eligible providers. Matchmaking selects the lowest eligible priority. A denied or missing primary can use an engineered fallback. Unrelated providers and both inter-cloud flags cannot escape a managed scope. Unsupported existing translation/QoS features remain unsupported.

Mutual TLS and certificate-bound identity tokens remain mandatory. Only PDE or Sysop can replace; PDE, LocalCloudManager or Sysop can query. PDE gains no general DSO administration right. ServiceRegistry reserves the new service definition for DSO. Native Rust permits PDE only the two required registry query operations. DSO's `plantRules.active` record is persisted by an atomic private JSON commit and survives restart. Existing native state files need no SQL migration. Java uses its separate table migration.

The Go deployment script regenerates pinned runtime origins and installs scoped policies; the Rust script targets the native candidate. Both preserve state. Legacy SimpleStore rows remain independent and are not imported or automatically deleted. Coordinated rollback must restore PDE and its backend together. Current live evidence covers priority zero, fallback, unavailable producers, both inter-cloud guards, restart persistence with PDE stopped, and deactivation. This does not claim complete Java equivalence or production load qualification.